INSTITUTO FEDERAL
Rondônia

PROGRAMAÇÃO COMPETITIVA

Fundamentos: Lógica, Estruturas e Algoritmos

SUMÁRIO

Ferramentas e Ambiente **3**
Primeiros passos na programação **3**
Exemplo de Problema 4
Dicas para começar 4
Estruturas de decisão **4**
O que são 4
Operadores relacionais e lógicos 5
Tipos principais 6
Fluxo de execução 7
Editor Online vs Editor Local **7**
Impacto no Aprendizado 8
Controle vs Abstração 8
IDEs e Ferramentas 8
Organização e Execução 9
Erros e Diagnóstico 9
Onde escrever e executar seu código na prática 9
Uso direto no navegador (sem instalação) 10
Ambiente no computador (uso mais completo) 10
Instalação e uso básico do VS Code 10
Executando seu primeiro código 10
Por que isso importa? 11
Síntese 11
Estruturas de Repetição **11**
Repetição com quantidade definida: for 12
Repetição com condição: while 13
Repetição com execução garantida: do...while 14
Estruturas de Dados Básicas **16**
Listas (Arrays) 16
Estruturas chave-valor (dicionários/objetos) 17
Estruturas de pilha (Stack) 19
Estruturas de fila (Queue) 20
Algoritmos Fundamentais **21**
Busca Linear 22
Busca com índice 23
Ordenação de dados 23
Recursão 24
Raciocínio algorítmico 25

Ferramentas e Ambiente

O **beecrowd** é um site onde você pode treinar programação resolvendo desafios. Funciona assim: você cria uma conta, escolhe um problema e escreve uma solução em forma de código. Depois, envia esse código para o sistema, que verifica se está correto.

Quando o sistema encontra algum problema, ele mostra mensagens que ajudam você a entender o que precisa melhorar:

- **WA (Wrong Answer)**: sua resposta não bate com a esperada.
- **TLE (Time Limit Exceeded)**: seu programa demorou demais para rodar.
- **CE (Compilation Error)**: o código tem erro de escrita e não conseguiu começar.
- **RE (Runtime Error)**: o programa começou, mas parou no meio do caminho.

Você pode escrever seus códigos direto no site (editor online) ou instalar programas no computador (editor local).

- O **editor online** é simples e rápido, mas tem menos recursos.
- O **editor local** precisa da instalação do **Python** e de uma **IDE** (como VS Code ou PyCharm), que oferecem mais ferramentas para testar e organizar seus programas.

O primeiro passo é resolver um problema guiado no beecrowd, que mostra como funciona o processo. Nesse momento, você aprende a lidar com **entrada e saída de dados** — ou seja, como o programa recebe informações e como deve mostrar os resultados.

Esse ciclo básico — escrever o código, enviar para o sistema e interpretar o resultado — é o que vai dar confiança para você praticar sozinho e enfrentar novos desafios.

Primeiros passos na programação

A programação competitiva é uma forma de treinar lógica e raciocínio usando códigos de computador. Nela, você recebe problemas que precisam ser resolvidos com rapidez e eficiência. É muito usada em competições, entrevistas de emprego e também como exercício para desenvolver o pensamento lógico.

O principal objetivo é aprender a pensar de forma estruturada. Cada problema exige que você analise a situação, crie um passo a passo (algoritmo) e depois transforme esse raciocínio em código. Isso ajuda a resolver problemas complexos e melhora a capacidade de raciocínio em outras áreas também.

Além de pensar corretamente, é preciso pensar rápido. A prática constante ensina a escrever códigos que funcionam bem e que rodam em pouco tempo, evitando desperdício de recursos.

Existem maratonas e olimpíadas de programação em que os participantes recebem vários problemas e precisam resolvê-los dentro de um tempo limitado. Essas competições simulam situações reais e são uma ótima forma de testar habilidades.

Exemplo de Problema

Um exemplo simples é verificar se um número é **par ou ímpar**.

Em código, isso pode ser feito assim:

```python
# Solicita ao usuário que digite um número
# input() retorna uma string, então usamos int() para converter para inteiro
numero = int(input("Digite um número: "))

# Verifica se o número é par
# O operador % (módulo) retorna o resto da divisão
# Se o resto da divisão por 2 for igual a 0, o número é par
if numero % 2 == 0:
    print("Par")  # Executa se a condição for verdadeira
else:
    print("Ímpar")  # Executa se a condição for falsa
```

Esse problema mostra como a lógica funciona: você recebe uma entrada (o número), aplica uma regra (verificar se sobra resto na divisão por 2) e gera uma saída (par ou ímpar).

Dicas para começar

- **Comece com problemas fáceis**: eles ajudam a entender a lógica básica.
- **Aprenda estruturas fundamentais**: como if (condições), for e while (repetições).
- **Treine todos os dias**: a prática constante é o que traz evolução.
- **Leia soluções de outros programadores**: isso mostra diferentes formas de pensar.
- **Use plataformas online**: como o beecrowd, que oferece problemas e corrige automaticamente suas soluções.

Estruturas de decisão

O que são

Estruturas de decisão são mecanismos fundamentais da programação que permitem ao programa escolher diferentes caminhos de execução dependendo de condições específicas. Elas funcionam como escolhas que o computador precisa tomar, semelhantes às decisões que fazemos no dia a dia. Imagine que você esteja indo para a escola: se estiver

chovendo, você leva guarda-chuva; se não estiver, sai sem ele. Esse raciocínio humano é exatamente o que o computador aprende a reproduzir. Sem estruturas de decisão, o programa seria apenas uma sequência fixa de instruções, sempre repetindo o mesmo comportamento, sem reagir ao ambiente ou às informações recebidas. Com elas, o código ganha inteligência, flexibilidade e capacidade de adaptação, tornando-se capaz de reagir a diferentes situações e se aproximando da forma como pensamos. Esse conceito é tão importante que está presente em praticamente todas as linguagens de programação, desde as mais simples até as mais complexas, e constitui a base para qualquer sistema que precise interagir com o usuário ou com dados externos.

Importância

Estruturas de decisão são essenciais porque dão **inteligência** ao sistema. Elas são a base da lógica em jogos, aplicativos e softwares em geral. Sem elas, não seria possível criar programas que se adaptam às condições do usuário ou do ambiente. Por isso, estão presentes em praticamente todas as linguagens de programação, como JavaScript, Python, C ou Java. Dominar esse conceito é o primeiro passo para criar sistemas inteligentes e funcionais, porque é a partir dele que o código deixa de ser apenas uma sequência de instruções e passa a reagir às situações.

Lógica booleana

Toda estrutura de decisão se apoia em um conceito chamado **lógica booleana**, que é a matemática do verdadeiro e falso. Em programação, uma condição sempre retorna um valor booleano: ou é verdadeira (true), ou é falsa (false). Por exemplo, a expressão idade >= 18 retorna verdadeiro se a idade for maior ou igual a 18, e retorna falso caso contrário. Esse mecanismo é o que permite ao computador avaliar situações e decidir qual caminho seguir. Sem compreender a lógica booleana, o estudante não consegue entender como o if funciona, porque é justamente esse resultado binário que determina se um bloco de código será executado ou não.

Operadores relacionais e lógicos

Para construir condições em programação, utilizamos operadores que permitem comparar valores ou combinar diferentes expressões lógicas. Esses operadores são fundamentais para que o programa tome decisões com base em dados.

Os operadores relacionais são responsáveis por comparar dois valores e sempre retornam um resultado booleano, ou seja, verdadeiro (**True**) ou falso (**False**). Os principais são: **< (menor que), > (maior que), <= (menor ou igual), >= (maior ou igual), == (igualdade) e != (diferença).**

Já os operadores lógicos permitem combinar múltiplas condições em uma única expressão. Em Python, eles são: **and** (equivalente ao "e"), **or** (equivalente ao "ou") e **not** (negação). O operador **and** exige que todas as condições sejam verdadeiras. O operador **or** exige que

pelo menos uma condição seja verdadeira. Já o operador **not** inverte o valor lógico de uma expressão.

Com esses operadores, é possível construir verificações mais completas. Por exemplo, podemos validar se uma pessoa tem idade suficiente e possui um documento válido ao mesmo tempo, criando regras mais próximas de situações reais.

Por exemplo:

```
# Exemplo de operadores relacionais
idade = 20  # variável que armazena um valor numérico

# Verifica se a idade é maior ou igual a 18
print(idade >= 18)  # Retorna True ou False


# Exemplo de operadores lógicos
possui_documento = True  # variável booleana

# Verifica duas condições ao mesmo tempo usando "and"
# Ambas precisam ser verdadeiras para entrar no if
if idade >= 18 and possui_documento:
    print("Pode entrar")
else:
    print("Não pode entrar")
```

Nesse caso, o programa só permite a entrada se as duas condições forem verdadeiras. Esse tipo de combinação é essencial para sistemas reais, que raramente dependem de apenas uma verificação simples.

Tipos principais

Existem três formas básicas de estruturar decisões em programação:

- **if** → significa "se". O programa verifica uma condição e, se ela for verdadeira, executa um bloco de instruções.
- **else** → significa "senão". O programa executa esse bloco quando o if não é verdadeiro.
- **else if** → significa "senão se". Permite testar várias condições em sequência, tornando o raciocínio mais completo.

Essas estruturas podem ser combinadas para criar sistemas mais complexos. Por exemplo, em um sistema de votação, podemos verificar a idade de uma pessoa: se for menor de 16 anos, não pode votar; se estiver entre 16 e 18, o voto é opcional; se tiver 18 ou

mais, o voto é obrigatório. Esse tipo de lógica mostra como o computador avalia diferentes cenários e escolhe o caminho adequado, exatamente como fazemos em situações reais.

Exemplo prático

Um exemplo simples é verificar a idade de uma pessoa para decidir sobre o voto:

```
idade = 16  # Define a variável idade com valor 16

if idade < 16:  # Verifica se a idade é menor que 16
    print("Não pode votar")  # Executa se a condição for verdadeira
elif idade >= 16 and idade < 18:  # Verifica se está entre 16 (inclusive) e 18 (exclusivo)
    print("Voto opcional")  # Executa se essa condição for verdadeira
else:  # Caso nenhuma das condições anteriores seja verdadeira
    print("Voto obrigatório")  # Executa para idade maior ou igual a 18
```

Nesse caso, o programa avalia a idade e escolhe qual mensagem mostrar. Esse tipo de lógica é usado em praticamente todos os sistemas, desde aplicativos simples até softwares complexos, porque permite que o programa se adapte às informações que recebe.

Fluxo de execução

O fluxo de execução em estruturas de decisão segue sempre uma ordem de cima para baixo. O programa avalia a primeira condição; se ela for verdadeira, executa o bloco correspondente e ignora os demais. Se for falsa, passa para a próxima condição, e assim sucessivamente. No caso do if...else if...else, apenas um bloco será executado, nunca mais de um. Isso significa que a ordem das condições é fundamental: colocar uma verificação mais ampla antes de uma mais específica pode impedir que o programa chegue ao resultado correto. Entender esse fluxo é essencial para evitar erros lógicos e garantir que o programa se comporte como esperado.

Editor Online vs Editor Local

A principal diferença entre editor online e editor local não é onde o código roda, mas **quem controla o ambiente**.

No editor online, o ambiente já está pronto. A execução do código, a entrada de dados e a validação acontecem dentro de uma estrutura controlada pela plataforma. Isso reduz a complexidade, mas também esconde parte do funcionamento real.

No editor local, o ambiente depende do usuário. O código é executado diretamente na máquina, os arquivos são organizados manualmente e a execução depende da configuração correta do sistema. Isso exige mais atenção, mas expõe como a programação realmente funciona fora de plataformas controladas. Essa diferença é essencial: um abstrai o funcionamento, o outro revela.

Impacto no Aprendizado

O editor online facilita o início porque elimina variáveis externas. O estudante foca no problema, na lógica e na sintaxe. Isso é útil nos primeiros contatos, onde o excesso de complexidade pode atrapalhar mais do que ajudar.

Por outro lado, esse modelo reduz o contato com aspectos importantes da prática real, como execução via terminal, organização de arquivos e comportamento do código fora de um ambiente controlado.

Já o editor local introduz essas variáveis desde o começo. Isso aumenta a dificuldade inicial, mas desenvolve uma compreensão mais completa. O estudante passa a lidar não apenas com o código, mas com o contexto onde ele roda.

Esse contato direto melhora a capacidade de diagnosticar erros, entender falhas e construir soluções mais consistentes.

Controle vs Abstração

Toda ferramenta de programação trabalha em algum nível de abstração. Quanto mais alto esse nível, menos detalhes o usuário precisa gerenciar.

O editor online opera em um nível alto de abstração: ele oculta configurações, padroniza entradas e controla a execução. Isso torna o uso mais simples, mas limita a visão do processo completo.

O editor local reduz esse nível de abstração. Ele exige que o usuário entenda como o código é executado, onde os arquivos estão e como o sistema responde. Em troca, oferece mais controle e previsibilidade.

Aprender programação envolve, inevitavelmente, sair da abstração e entender o que está acontecendo por trás.

IDEs e Ferramentas

Um ambiente de programação não se resume ao editor. IDEs (Ambientes de Desenvolvimento Integrado) ampliam esse conceito ao reunir múltiplas ferramentas em um único espaço.

Uma IDE organiza o fluxo de desenvolvimento: edição de código, execução, navegação entre arquivos e identificação de erros. Ela não resolve problemas lógicos, mas reduz o esforço mecânico envolvido no processo.

Isso permite que o foco seja deslocado para o que realmente importa: estrutura do código, clareza da solução e correção dos resultados.

Ferramentas bem utilizadas aumentam produtividade, mas não substituem entendimento.

Organização e Execução

Programar envolve mais do que escrever código isolado. Mesmo em exercícios simples, existe uma estrutura mínima: arquivos, entradas, saídas e forma de execução.

No ambiente local, essa estrutura fica explícita. O estudante precisa entender:

- onde o arquivo está salvo
- como executar o código
- como o programa recebe entrada
- como o resultado é exibido

Esse processo desenvolve uma visão mais concreta do funcionamento da programação. Em vez de apenas "rodar código", o estudante passa a entender o ciclo completo de execução.

Erros e Diagnóstico

Um dos pontos mais importantes do ambiente é a forma como ele expõe erros.

Em ambientes online, os erros costumam ser simplificados ou padronizados. Isso ajuda no início, mas pode esconder detalhes importantes.

No ambiente local, os erros aparecem de forma mais "crua". Isso exige interpretação, mas desenvolve uma habilidade essencial: **diagnosticar problemas com base em mensagens do sistema**.

Aprender a ler erros, identificar causas e testar correções é uma das competências centrais na programação.

Onde escrever e executar seu código na prática

Depois de entender como o ambiente influencia o aprendizado, o próximo passo é saber onde você vai, de fato, escrever e testar seus códigos.

Na prática, existem dois caminhos principais: usar ferramentas online ou configurar um ambiente no próprio computador.

Uso direto no navegador (sem instalação)

Algumas plataformas permitem escrever e executar código imediatamente, sem precisar instalar nada.

Exemplos:

- OnlineGDB → https://www.onlinegdb.com
- Replit → https://replit.com

Essas ferramentas são úteis para:

- testar pequenos códigos
- focar apenas na lógica

O funcionamento é simples: você escreve o código, executa e analisa o resultado. Tudo acontece dentro da própria plataforma.

Ambiente no computador (uso mais completo)

Para ter mais controle e evoluir na prática, o ideal é usar um editor local.

Uma das opções mais utilizadas é o:

👉 Visual Studio Code (VS Code)
Download: https://code.visualstudio.com/

Instalação e uso básico do VS Code

Após acessar o site:

1. Baixe a versão para seu sistema operacional
2. Instale normalmente (próximo → próximo → concluir)
3. Abra o VS Code

Executando seu primeiro código

Fluxo básico:

1. Crie uma pasta no seu computador (ex: meus_codigos)
2. Abra essa pasta no VS Code
3. Crie um arquivo (ex: teste.py, main.cpp, etc.)
4. Escreva um código simples
5. Abra o terminal dentro do VS Code
6. Execute o arquivo

Por que isso importa?

Ao usar um ambiente local, você passa a entender:

- como arquivos são organizados
- como o código é executado de verdade
- como o sistema responde ao seu programa

Isso desenvolve autonomia e reduz dependência de plataformas prontas.

Síntese

O ambiente de programação não é apenas um detalhe técnico — ele define o nível de controle, o tipo de erro que você enfrenta e a profundidade do seu entendimento.

- Editor online → mais abstração, menos controle
- Editor local → menos abstração, mais controle

Compreender essa diferença evita dependência de ferramentas e desenvolve autonomia desde cedo.

Programar bem não depende do ambiente, mas entender o ambiente acelera o caminho até isso.

Estruturas de Repetição

Programar não envolve apenas executar instruções uma única vez. Em muitos problemas, é necessário repetir ações, processar conjuntos de dados ou executar etapas sucessivas até atingir um determinado resultado. Estruturas de repetição existem para controlar esse tipo de comportamento.

Elas permitem que um bloco de código seja executado múltiplas vezes, com base em uma condição ou em um intervalo definido. Sem esse recurso, até tarefas simples exigiriam repetição manual de código, tornando os programas extensos, difíceis de manter e propensos a erros.

A lógica de repetição sempre envolve três elementos fundamentais: início, condição e atualização. O programa começa em um estado inicial, executa um conjunto de instruções e, ao final de cada ciclo, verifica se deve continuar ou parar.

Repetição com quantidade definida: for

A estrutura **for** é utilizada quando o número de repetições é conhecido ou pode ser controlado diretamente.

Exemplo:

```
# Estrutura de repetição FOR (contagem simples)

for i in range(1, 6):  # range(1, 6) gera números de 1 até 5
    print(i)  # Imprime o valor atual de i
```

Explicação:

• A variável **i** assume valores de 1 até 5
• O valor final (6) não é incluído no intervalo
• A cada repetição, o valor atual de **i** é impresso
• O controle do loop é automático, sem necessidade de atualização manual

Exemplo prático:

```
# FOR com acumulação (somando valores)

soma = 0  # Inicializa a variável soma

for i in range(1, 6):  # Loop de 1 até 5
    soma = soma + i  # Soma acumulativa

print(soma)  # Resultado final da soma
```

Explicação:

• A variável **soma** começa com valor 0
• A cada iteração, o valor de **i** é somado
• O resultado final será 15
• Esse padrão representa um processo de acumulação

Repetição com condição: while

A estrutura **while** executa um bloco de código enquanto uma condição for verdadeira.

Exemplo:

```
# Estrutura WHILE com contador

contador = 1  # Inicializa o contador

while contador <= 5:  # Enquanto contador for <= 5
    print(contador)  # Imprime o valor atual
    contador = contador + 1  # Incrementa o contador
```

Explicação:

• O loop começa com **contador = 1**
• A condição é verificada antes de cada execução
• O valor é exibido e depois atualizado
• Quando o valor chega a 6, o loop é encerrado

Exemplo prático:

```
# WHILE com entrada de dados

numero = int(input())  # Lê um número inteiro

while numero != 0:  # Continua até o usuário digitar 0
    print("Valor:", numero)  # Mostra o número
    numero = int(input())  # Lê novo valor
```

Explicação:

• O programa continua enquanto o valor for diferente de 0
• O valor 0 funciona como condição de parada
• O usuário controla o fim da repetição
• Esse padrão é comum em leitura de dados

Erro crítico: loop infinito

```
# Exemplo com erro lógico (loop infinito)

contador = 1  # Inicializa o contador

while contador <= 5:  # Condição sempre verdadeira
    print(contador)  # Imprime o valor
    # Falta incrementar o contador → loop infinito
```

Problema:

• O valor de **contador** nunca é atualizado
• A condição nunca se torna falsa
• O programa entra em execução infinita

Repetição com execução garantida: do...while

Essa estrutura garante que o bloco seja executado pelo menos uma vez, independentemente da condição inicial. Isso acontece porque a verificação da condição é feita somente depois da primeira execução do bloco. Ou seja, primeiro o código roda, depois o programa decide se deve repetir ou não. Esse comportamento é útil em situações onde pelo menos uma execução é obrigatória, como menus interativos ou leitura inicial de dados. O bloco executa antes da verificação; depois disso, a condição é avaliada; se ainda for verdadeira, o loop continua.

Observação: Python não possui a estrutura do...while nativamente. Por isso, abaixo há exemplos que estão em JavaScript, apenas para demonstrar esse tipo específico de repetição. Mas, em Python é possível simular o comportamento de um do… while, logo abaixo do exemplo em JavaScript há uma simulação.

```javascript
// Estrutura DO...WHILE em JavaScript

let contador = 1; // Inicializa a variável contador com valor 1

do {
    console.log(contador); // Executa o bloco primeiro, imprimindo o valor atual
    contador++; // Incrementa o contador a cada repetição
} while (contador <= 5); // Após executar o bloco, verifica a condição para continuar
```

```python
# Estrutura adaptada em Python para simular um DO...WHILE

contador = 1  # Inicializa a variável contador com valor 1

while True:
    print(contador)  # Executa o bloco primeiro, imprimindo o valor atual
    contador += 1  # Incrementa o contador a cada repetição

    if contador > 5:  # Após executar o bloco, verifica a condição de parada
        break  # Encerra o laço quando a condição não for mais satisfeita
```

Explicação:

• O bloco executa antes da verificação
• Depois disso, a condição é avaliada
• Se ainda for verdadeira, o loop continua

Exemplo prático:

```javascript
let opcao; // Declara a variável que armazenará a escolha do usuário

do {
    console.log("1 - Continuar"); // Mostra a opção de continuar
    console.log("0 - Sair"); // Mostra a opção de sair

    opcao = Number(prompt()); // Lê a entrada do usuário e converte para número

} while (opcao !== 0); // Continua repetindo enquanto o usuário não digitar 0
```

```python
# Simulação do comportamento de um do...while em Python (exemplo prático)

while True:
    print("1 - Continuar")  # Mostra a opção de continuar
    print("0 - Sair")  # Mostra a opção de sair

    opcao = int(input())  # Lê a entrada do usuário e converte para inteiro

    if opcao == 0:  # Após executar o bloco, verifica a condição de parada
        break  # Encerra o laço quando o usuário escolher sair
```

Explicação:

• O menu aparece pelo menos uma vez
• O usuário define quando parar
• Ideal para sistemas interativos

Raciocínio em repetições

Ao utilizar estruturas de repetição, é necessário acompanhar:

• Valor atual das variáveis
• Condição de continuidade
• Momento de parada
• Evolução do processo a cada passo

Estruturas de repetição permitem que o programa execute processos ao longo do tempo, em vez de apenas instruções isoladas. Elas são essenciais para transformar lógica simples em soluções completas.

Dominar esse conceito significa entender não apenas como repetir, mas como controlar corretamente o fluxo de execução.

Estruturas de Dados Básicas

Resolver problemas em programação não depende apenas de lógica, mas também de como os dados são organizados. Estruturas de dados são formas de armazenar, acessar e manipular informações de maneira eficiente dentro de um programa.

Sem uma organização adequada, o código tende a ficar confuso, repetitivo e difícil de escalar. Com a estrutura correta, o mesmo problema pode ser resolvido de forma mais simples, clara e eficiente.

Em essência, estruturas de dados respondem a uma pergunta fundamental: **como representar informações dentro do programa?**

Listas (Arrays)

A estrutura mais básica é a lista (ou array). Ela permite armazenar vários valores em uma única variável, organizados por posição.

Exemplo:

```
# Acesso a elementos de uma lista por índice

numeros = [10, 20, 30, 40]  # Cria uma lista com 4 elementos
print(numeros[0])  # Acessa o primeiro elemento (índice 0)
print(numeros[2])  # Acessa o terceiro elemento (índice 2)
```

Explicação:

• A lista armazena múltiplos valores em ordem
• Cada elemento possui um índice (posição)
• O índice começa em 0
• numeros[0] retorna 10
• numeros[2] retorna 30

Exemplo prático:

```
# Soma de elementos usando for

soma = 0  # Inicializa acumulador
numeros = [1, 2, 3, 4, 5]  # Lista de números

for n in numeros:  # Percorre cada elemento da lista
    soma += n  # Soma o valor atual ao acumulador

print(soma)  # Exibe o resultado final
```

Explicação:

• A repetição percorre todos os elementos da lista
• A variável **n** representa o valor atual
• Cada valor é somado à variável **soma**
• O resultado final será 15

Operações comuns com listas

• Acessar elementos por índice
• Percorrer todos os elementos
• Modificar valores
• Adicionar novos elementos
• Remover elementos

Essas operações aparecem em praticamente qualquer programa.

Erros comuns com listas

• Tentar acessar uma posição inexistente (erro de índice)
• Esquecer que o índice começa em 0
• Confundir posição com valor

Estruturas chave-valor (dicionários/objetos)

Diferente das listas, onde o acesso é feito por posição, estruturas chave-valor permitem acessar dados por meio de nomes.

Exemplo:

```python
# Uso de dicionário (estrutura chave-valor)

pessoa = {
    "nome": "Ana",  # Chave "nome" associada ao valor "Ana"
    "idade": 18     # Chave "idade" associada ao valor 18
}

print(pessoa["nome"])  # Acessa o valor da chave "nome"
print(pessoa["idade"])  # Acessa o valor da chave "idade"
```

Explicação:

• Os dados são organizados em pares (chave → valor)
• "nome" e "idade" são as chaves
• O acesso é feito pelo nome da chave
• pessoa["nome"] retorna "Ana"

Exemplo prático:

```python
# Condição baseada em valor de dicionário

aluno = {
    "nome": "Carlos",  # Nome do aluno
    "nota": 7.5        # Nota do aluno
}

if aluno["nota"] >= 7:  # Verifica se a nota é maior ou igual a 7
    print("Aprovado")  # Caso verdadeiro
else:
    print("Reprovado")  # Caso contrário
```

Explicação:

• O valor da nota é acessado pela chave
• A decisão é feita com base nesse valor
• Estruturas chave-valor facilitam representar dados reais

Quando usar listas ou dicionários

• Use lista quando os dados são sequenciais
• Use chave-valor quando os dados têm significado específico

Estruturas de pilha (Stack)

A pilha segue o princípio **LIFO (Last In, First Out)**, ou seja, o último elemento inserido é o primeiro a sair.

Exemplo:

```python
# Estrutura de pilha (LIFO)

pilha = []  # Lista vazia representando a pilha

pilha.append(10)  # Insere 10 no topo
pilha.append(20)  # Insere 20 no topo
pilha.append(30)  # Insere 30 no topo

print(pilha.pop())  # Remove e retorna o último elemento (30)
print(pilha)  # Mostra o estado atual da pilha
```

Explicação:

• append adiciona elementos no topo
• pop remove o último elemento inserido
• O valor removido será 30
• A pilha restante será [10, 20]

Aplicação prática de pilha

• Controle de histórico (ex: botão “voltar”)
• Avaliação de expressões
• Processos que precisam desfazer ações

Estruturas de fila (Queue)

A fila segue o princípio **FIFO (First In, First Out)**, ou seja, o primeiro elemento inserido é o primeiro a sair.

Exemplo:

```
# Estrutura de fila (FIFO)

from collections import deque  # Importa estrutura otimizada para fila

fila = deque()  # Cria uma fila vazia

fila.append(10)  # Adiciona 10 ao final
fila.append(20)  # Adiciona 20 ao final
fila.append(30)  # Adiciona 30 ao final

print(fila.popleft())  # Remove e retorna o primeiro elemento (10)
print(fila)  # Mostra o estado atual da fila
```

Explicação:

• append adiciona no final
• popleft remove o primeiro elemento
• O valor removido será 10
• A fila restante será [20, 30]

Aplicação prática de fila

• Sistemas de atendimento
• Processamento de tarefas
• Simulações de ordem de chegada

Comparação entre estruturas

• Lista → sequência ordenada de valores
• Dicionário → acesso por chave
• Pilha → último a entrar, primeiro a sair
• Fila → primeiro a entrar, primeiro a sair

Cada estrutura resolve um tipo diferente de problema.

Raciocínio com estruturas de dados

Escolher a estrutura correta influencia diretamente:

• Clareza do código
• Facilidade de manutenção
• Eficiência da solução

Uma escolha inadequada pode tornar um problema simples muito mais complexo do que deveria ser.

Aplicações práticas

Estruturas de dados são usadas para:

• Armazenar entradas do usuário
• Representar informações do mundo real
• Processar grandes quantidades de dados
• Resolver problemas de busca e organização

Estruturas de dados são a base da organização em programação. Elas não apenas armazenam informações, mas definem como essas informações serão manipuladas.

Dominar esse conceito significa deixar de apenas "usar variáveis" e passar a estruturar dados de forma consciente e eficiente.

Algoritmos Fundamentais

Um algoritmo é uma sequência finita de passos bem definidos para resolver um problema. Em programação, ele representa a lógica por trás do código — ou seja, antes de escrever qualquer linha, é necessário saber **como o problema será resolvido**.

Saber programar não é apenas conhecer uma linguagem, mas saber construir algoritmos corretos, claros e eficientes. Dois programas podem resolver o mesmo problema, mas com desempenhos completamente diferentes dependendo do algoritmo utilizado.

Algoritmos fundamentais são aqueles que aparecem com frequência e servem como base para problemas mais complexos. Entre eles, destacam-se busca, ordenação e recursão.

Busca Linear

A busca linear é o algoritmo mais simples para encontrar um elemento em uma lista. Ele percorre os dados um por um até encontrar o valor desejado ou chegar ao final.

Exemplo:

```
# Busca simples (verifica existência)

numeros = [3, 7, 2, 9, 5]  # Lista de números
alvo = 9  # Valor que queremos encontrar
encontrado = False  # Variável de controle

for n in numeros:  # Percorre a lista
    if n == alvo:  # Verifica se encontrou o valor
        encontrado = True  # Marca como encontrado
        break  # Interrompe o loop

print(encontrado)  # Mostra se encontrou ou não
```

Explicação:

• O algoritmo percorre todos os elementos da lista
• A cada passo, compara o valor atual com o alvo
• Se encontrar, altera a variável e interrompe o loop
• Se não encontrar, termina normalmente

Limitação da busca linear

• Precisa verificar elemento por elemento
• Pode ser lenta para listas grandes
• Não aproveita nenhuma organização prévia dos dados

Mesmo assim, é simples e funciona em qualquer situação.

Busca com índice

Exemplo:

```python
# Busca com posição (retorna índice)

numeros = [3, 7, 2, 9, 5]  # Lista de números
alvo = 9  # Valor procurado
posicao = -1  # Valor padrão caso não encontre

for i in range(len(numeros)):  # Percorre índices da lista
    if numeros[i] == alvo:  # Compara elemento atual com alvo
        posicao = i  # Armazena a posição encontrada
        break  # Interrompe o loop

print(posicao)  # Mostra a posição do elemento
```

Explicação:

• O loop percorre os índices da lista
• numeros[i] acessa cada elemento
• Quando encontra o valor, armazena a posição
• Retorna -1 caso não encontre

Ordenação de dados

Ordenar significa reorganizar os elementos de uma lista em uma determinada ordem (crescente ou decrescente). Isso facilita buscas, comparações e análise de dados.

Um dos algoritmos mais simples é o **Bubble Sort**.

Exemplo:

```python
# Bubble Sort (ordenando uma lista)

numeros = [5, 3, 8, 1]  # Lista inicial desordenada

for i in range(len(numeros)):  # Controla quantas vezes vamos percorrer a lista
    for j in range(len(numeros) - 1):  # Percorre comparando elementos vizinhos
        if numeros[j] > numeros[j + 1]:  # Se o elemento atual for maior que o próximo
            # Troca de posição entre os elementos
            numeros[j], numeros[j + 1] = numeros[j + 1], numeros[j]

print(numeros)  # Exibe a lista ordenada
```

Explicação:

• O algoritmo percorre a lista várias vezes
• Compara elementos vizinhos
• Troca quando estão fora de ordem
• A cada repetição, os maiores valores “sobem”

Limitações do Bubble Sort

• Ineficiente para grandes volumes de dados
• Faz muitas comparações desnecessárias
• Usado mais para aprendizado do que na prática

Recursão

Recursão é uma técnica onde uma função chama a si mesma para resolver um problema. Em vez de resolver tudo de uma vez, o problema é dividido em partes menores.

Exemplo: fatorial

```
# Função recursiva para calcular fatorial

def fatorial(n):  # Define a função fatorial
    if n == 0:  # Caso base: fatorial de 0 é 1
        return 1
    return n * fatorial(n - 1)  # Chamada recursiva reduzindo o problema

print(fatorial(5))  # Calcula 5! = 5*4*3*2*1
```

Explicação:

• O caso base é quando n == 0
• A função chama a si mesma com n - 1
• O resultado é construído multiplicando os valores
• fatorial(5) = 5 × 4 × 3 × 2 × 1

Importância do caso base

Sem uma condição de parada, a recursão nunca termina.

Exemplo incorreto:

```
# Exemplo de recursão com erro (sem condição de parada)

def erro(n):  # Define uma função recursiva incorreta
    return erro(n - 1)  # Chama a si mesma indefinidamente
```

Esse código gera erro porque nunca atinge um fim.

Iteração vs Recursão

• Iteração usa loops (for, while)
• Recursão usa chamadas de função
• Ambos resolvem problemas semelhantes
• Recursão pode ser mais intuitiva, mas consome mais memória

Eficiência de algoritmos

Nem todo algoritmo é igualmente eficiente. Alguns resolvem o problema rapidamente, outros podem levar muito mais tempo.

Fatores importantes:

• Número de operações realizadas
• Tamanho da entrada de dados
• Organização dos dados

Um algoritmo eficiente faz menos trabalho para chegar ao mesmo resultado.

Raciocínio algorítmico

Antes de programar, é importante pensar:

• Qual é o problema?
• Quais são as entradas?
• Qual deve ser a saída?
• Quais passos levam da entrada até a saída?

Esse processo evita erros e reduz retrabalho.

Aplicações práticas

Algoritmos são usados para:

• Buscar informações
• Organizar dados
• Resolver problemas matemáticos
• Processar entradas do usuário
• Otimizar sistemas

Eles estão presentes em praticamente todos os programas.

Algoritmos são o núcleo da programação. Linguagens mudam, ferramentas evoluem, mas a capacidade de estruturar soluções lógicas permanece essencial. Dominar algoritmos fundamentais é o que permite evoluir de quem apenas escreve código para quem realmente resolve problemas.